

Progetto TAP 2026: Catania Airport Real-Time Radar

Architettura a microservizi disaccoppiata con Machine Learning predittivo locale

Presentato per il corso Technologies for Advanced Programming 2025/2026

La Missione: Dal Dato Grezzo alla Decisione



IL PROBLEMA

Voli soggetti a **ritardi imprevisti**. Dati aeroportuali destrutturati, intrappolati nel DOM del sito web web della SAC (Società Aeroporto Catania) senza API ufficiali pubbliche.



LA SOLUZIONE

Pipeline Data Engineering end-to-end. Estrazione in tempo reale, brokeraggio in-memory, inferenza ML in locale su CPU e storage vettoriale indicizzato.



IL VALORE

Departure Board interattivo. Rischio ritardo calcolato semanticamente per singola rotazione, a disposizione degli operatori in zero-latenza.

La Missione: Dal Dato Grezzo alla Decisione





LA SOLUZIONE

Pipeline Data Engineering end-to-end. Estrazione in tempo reale, brokeraggio in-memory, inferenza ML in locale su CPU e storage vettoriale indicizzato.



IL VALORE

Departure Board interattivo. Rischio ritardo calcolato semanticamente per singola rotazione, a disposizione degli operatori in zero-latenza.

La Missione: Dal Dato Grezzo alla Decisione



IL PROBLEMA

Voli soggetti a **ritardi imprevisti**. Dati aeroportuali destrutturati, intrappolati nel DOM del sito web web della SAC (Società Aeroporto Catania) senza API ufficiali pubbliche.





IL VALORE

Departure Board interattivo. Rischio ritardo calcolato semanticamente per singola rotazione, a disposizione degli operatori in zero-latenza.

La Missione: Dal Dato Grezzo alla Decisione



IL PROBLEMA

Voli soggetti a **ritardi imprevisti**. Dati aeroportuali destrutturati, intrappolati nel DOM del sito web web della SAC (Società Aeroporto Catania) senza API ufficiali pubbliche.



LA SOLUZIONE

Pipeline Data Engineering end-to-end. Estrazione in tempo reale, brokeraggio in-memory, inferenza ML in locale su CPU e storage vettoriale indicizzato.



Architectural Blueprint: Ecosistema a Microservizi

1. PRODUTTORE
Ingestion



Scraping asincrono dal Web (SAC)

2. BROKER
Streaming



Redis (Cuscinetto in-memory)

3. CONSUMATORE
Processing & AI



Python + Ollama (Llama 3.2:1b)

4. MEMORIA
Storage



Elasticsearch (Salvataggio JSON complessi)

5. VISIONE
DataViz



Grafana (Control Room finale)

Nota Architetturale: Flusso dati strettamente unidirezionale, containerizzato e 100% disaccoppiato. Nessun modulo attende cronicamente l'altro.

Ingestion Stealth: Estrazione Asincrona

```
async_playwright()
wait_until='domcontentloaded'
evaluate() -> estrazione innerText dal DOM
```

- Bypass dei blocchi HTTP standard tramite browser automation headless.
- Caricamento rapido (domcontentloaded) per evitare freeze di rete.
- Parsing robusto del DOM senza API ufficiali.



Il **Produttore** pubblica senza sosta sulla lista 'coda_voli_catania' in Redis e si dimentica del dato, garantendo massima velocità di scraping.

Scelta Architetturale: Il Disaccoppiamento

	Apache Kafka	Redis
Latenza	Bassa	Praticamente Nulla (In-Memory)
Consumo RAM	Altissimo / Bloated	Ultraleggero (~15MB base)
Persistenza	Nativa su disco, complessa	Opzionale tramite Snapshotting
Setup & DevOps	Complesso (Zookeeper/KRaft)	Singolo container Plug&Play

VERDETTO TECNICO: Redis è stato scelto come perfetto cuscinetto in-memory. Evita la perdita di dati durante i picchi di scraping e disaccoppia fluidamente lo scraper dal motore IA, senza saturare le limitate risorse hardware a disposizione.

L’Intelligenza Centrale: Il Consumatore

- Estrazione bloccante (brpop(‘coda_voli_catania’)) per consumare CPU solo all’arrivo di nuovi messaggi.
- Parsing Identitario: Mapping automatico di loghi aziendali tramite regex (es. ‘FR’ -> Ryanair) prima dell’invio all’IA.



> “Asincronia Sicura: Nessun blocco di rete. Il consumatore lavora al proprio ritmo, disaccoppiando l’I/O di rete (lo scraping) dall’elaborazione CPU-bound (il Machine Learning). Se l’LLM è lento, Redis accumula semplicemente i messaggi in coda.

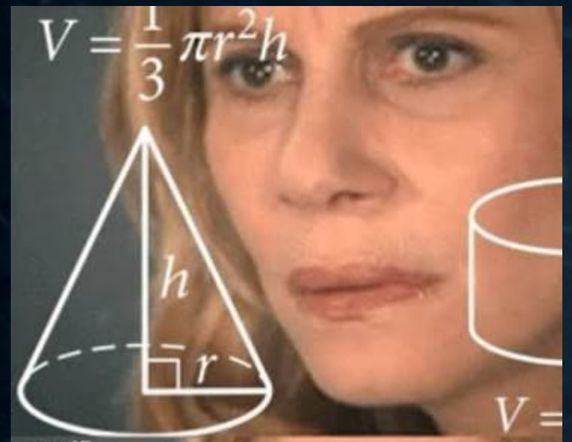
Analisi Predittiva Locale (Deep Dive Llama 3.2)

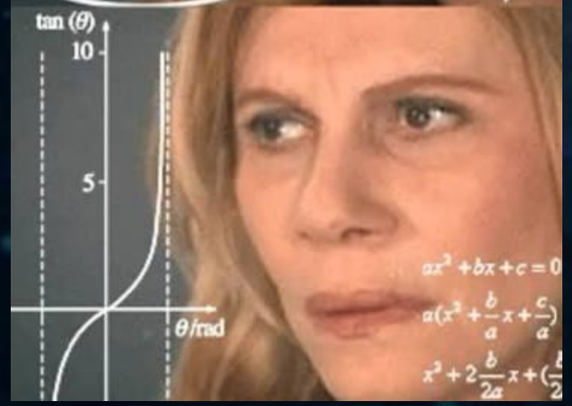
100% Air-gapped.
Privacy totale, nessun dato inviato a server esterni (No API OpenAI/Google).

Orario Schedulato vs Attuale

Storicità Rotazione Aereo

Tratta Specifica





Input

Processing

Output



Llama 3.2:1b

1 Miliardo di parametri. Ottimizzato anti-OOM killer per inferenza CPU pura.

```
{
  'rischio_ritardo': 'Alto',
  'ragionamento': 'Rotazione in ritardo
da volo precedente da Milano Malpensa'
}
```

Storage Dinamico Time-Series



```
{
  "volo": {
    "analisi_ia": {
      "predizione": {
        "predizione": "Alto",
        "dettagli_tempo": {
          "origin": "61",
          "time": "20212h"
        },
        "origini": {
          "origi": "Milano",
          "destinazione": "Roma",
          "temp": "Alto"
        },
        "destinazione": {
          "storicità": "Alto"
        }
      }
    }
```



NoSQL Flessibile

Accetta nativamente i JSON nidificati e complessi prodotti dall'LLM senza definire tabelle o schemi rigidi SQL.

Indicizzazione

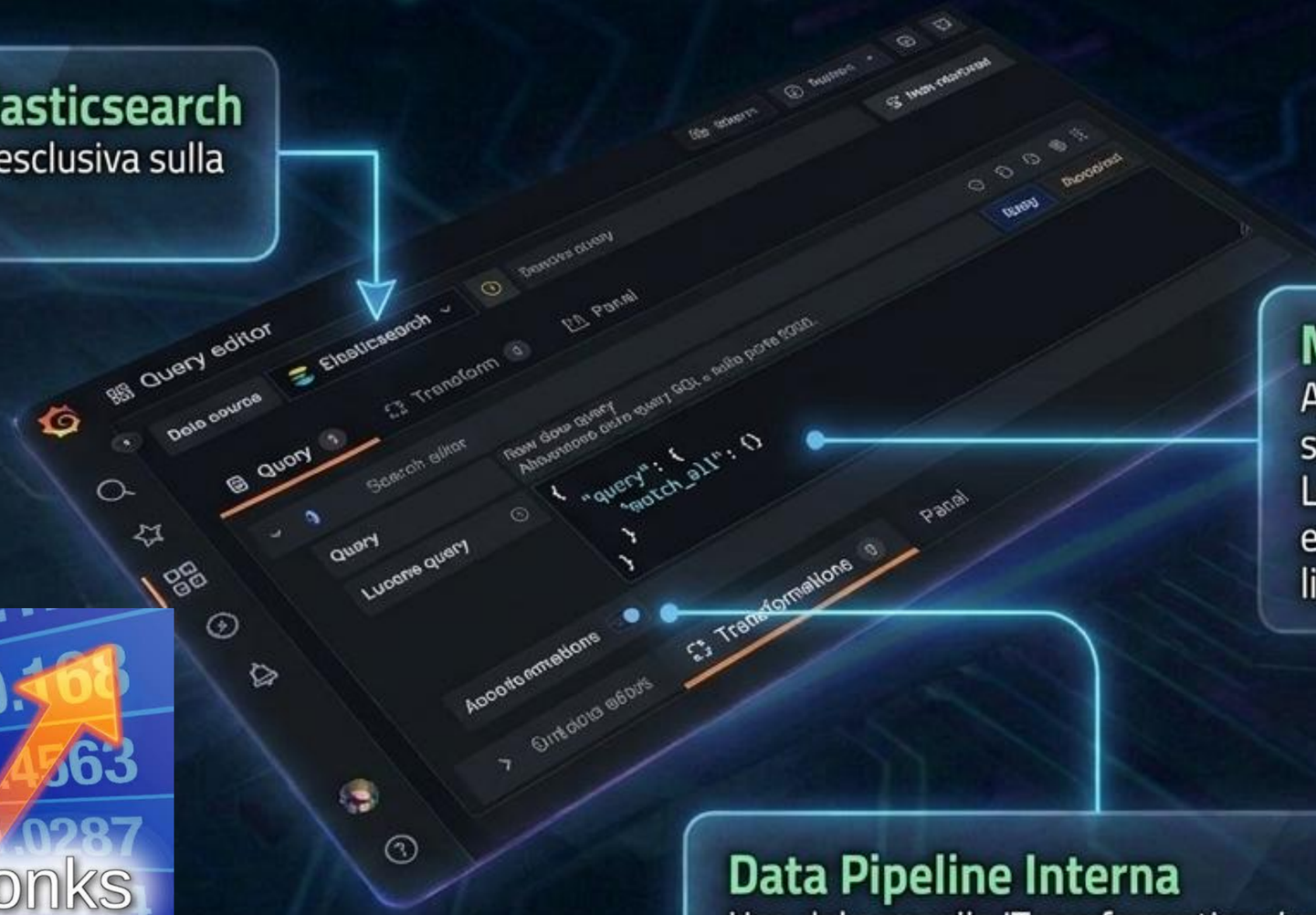
Salvataggio automatico sull'indice 'voli_analizzati', ottimizzato per query ultra-veloci.

Timezone Injection

Gestione critica del campo '@tirnestamp' tramite Python ('datetime.now().astimezone().isoformat()') per allineare perfettamente il fuso orario locale in fase di plottaggio su Grafana.

The Control Room: Setup Grafana

Connessione Elasticsearch
Integrazione nativa esclusiva sulla porta 9200.



Metriche 'Raw Document'
Abbandono delle query SQL standard. Utilizzo della sintassi Lucene per istruire Grafana a estrarre i JSON completi anziché limitarsi ai conteggi numerici.



Data Pipeline Interna
Uso del pannello 'Transformations' per mappare dinamicamente campi complessi in semplici righe e colonne da visualizzare.

Data Insights: Il Dominio delle Compagnie



Nota Tecnica: Applicazione di Value Mappings e Thresholds cromatici nativi in Grafana per enfatizzare le densità di traffico critiche.
I dati sono estratti dinamicamente tramite aggregazione sul campo testuale indicizzato ('compagnia.keyword').

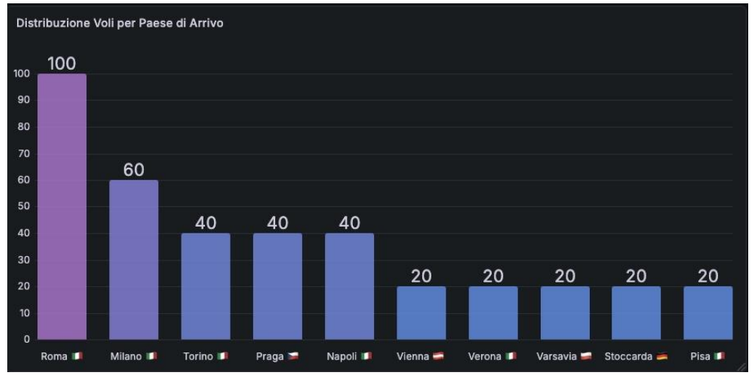
Mappa Operativa: Destinazioni e Status

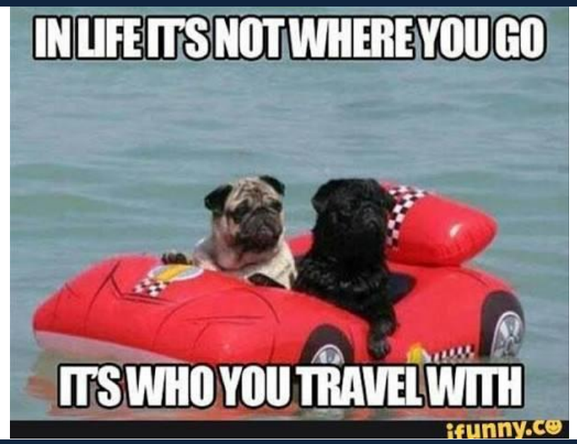


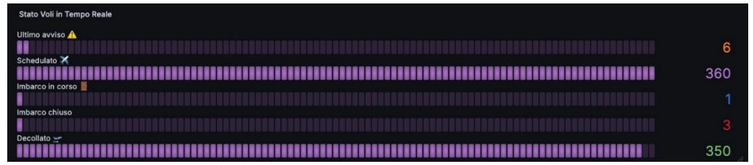


Destinazioni Principali
Utilizzo di un grafico a barre con "Value Mappings" per visualizzare le destinazioni, con un gradiente di colore per evidenziare le frequenze.

Stato Operativo
Visualizzazione dinamica dello stato dei voli tramite grafico a ciambella, con "Value Mappings" semantici (Testo -> Colore + Icona) per "Schedulato", "Decollato", "Imbarco in corso" e "Ultimo avviso".







UI/UX: Ricreare il Departure Board

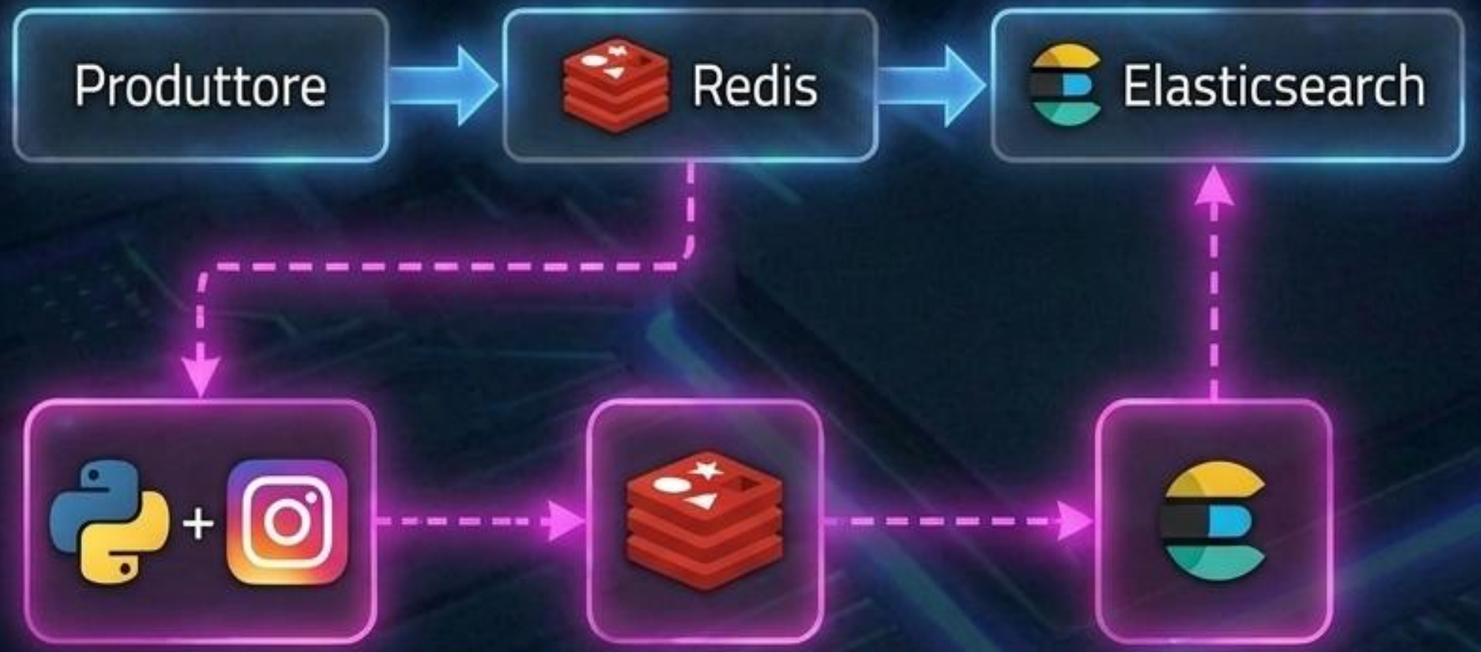

```
{"data": {
  "airline": "RYR",
  "flight_no": "FR123",
  "destination": "FCO",
  "sched_time": "2024-08-
15T14:00:00Z",
  "rischio_ritardo":
"ALTO_RITARDO",
  "note_ai": "Rischio critico per
congestione spazio aereo. Ritardo
stimato +90 min.",
  "logo_url": "/assets/ryanair.png"
}}
```




Compagnia	Volo	Destinazione	Orario	Stato	Note AI
Ryanair	FR123	Roma Fiumicino	16:00	⚠ Alto Ritardo	Rischio critico: Congestione, +90 min.

- **1. Organize fields:** Ridenominazione pulita dei campi macchina (‘rischio_ritardo’ -> ‘Stato’).
- **2. Field Overrides:** Regole dinamiche di background color per i badge di stato (Rosso/Verde).
- **3. Logo Injection:** Utilizzo di regex per renderizzare l’URL immagine affianco al nome compagnia.

Ecosistema Scalabile: Scraping Social



Playwright
(Social Scraper SAC)

Estrazione
comunicati ufficiali.

Nuova Coda
Dedicata
(Bypass LLM)

Indice
“instagram_cta”

Grafana

Comunicato SAC: Allerta cenere vulcanica. Sospensione voli temporanea.

“Risultato:
Le comunicazioni istituzionali sui social vengono iniettate in zero-latenza direttamente accanto ai dati dei voli sulla stessa dashboard Grafana, senza intaccare i microservizi esistenti.

Superamento delle Sfide: Hardware vs IA



Crash di Sistema (OOM Killer)

L'utilizzo del modello standard Llama 3 (8B parametri) richiedeva ~5GB di memoria.

Su un hardware limitato a 7GB di RAM totali, l'OOM Killer del sistema operativo terminava brutalmente il container Docker del consumatore per saturazione memoria.



Ottimizzazione Estrema

1. **Pivot Strategico**: Passaggio al modello edge 'Llama 3.2:1b' (1 miliardo parametri), ultraleggero ma semanticamente capace.
2. **Daemon Isolato**: Modifica del gestore processi ('nohup ollama serve &') per slegare il demone IA dalla shell bloccante.
3. **Code Asincrone**: Il ciclo bloccante 'brpop' si adatta dinamicamente al throughput della CPU locale, impedendo la perdita di dati anche in caso di rallentamenti dell'inferenza.

Syllabus Coverage & Conclusione

- [x] **Ingestion:** Web Scraping Asincrono Avanzato (Playwright Python).
- [x] **Streaming / Message Broker:** Code Pub/Sub in-memory (Redis).
- [x] **Processing & ML:** Inferenza NLP 100% in locale (Ollama / Llama 3.2:1b).
- [x] **Storage & Indexing:** Archiviazione NoSQL per serie temporali (Elasticsearch).
- [x] **Visualization:** Dashboard live con mapping dinamici e sintassi Lucene (Grafana).

Architettura 100% containerizzata, disaccoppiata e pronta per il deployment scalabile su cluster Kubernetes o ambienti Cloud.

Vincenzo Biondi [1000045330]
Fabio Scirè [1000057738]

Grazie per l'attenzione.